Jenkins — Complete Notes

Automation server · Build, test & deploy with pipelines

A reader-friendly, all-in-one reference for Jenkins.

---

1. 🤖 What is Jenkins

Jenkins is an **open-source automation server** for building, testing, and deploying software — the veteran, **self-hosted** CI/CD tool. You define pipelines as code, and Jenkins runs them on triggers (a commit, a schedule, a webhook), automating everything from compile to deploy. Its huge **plugin** ecosystem lets it integrate with almost any tool.

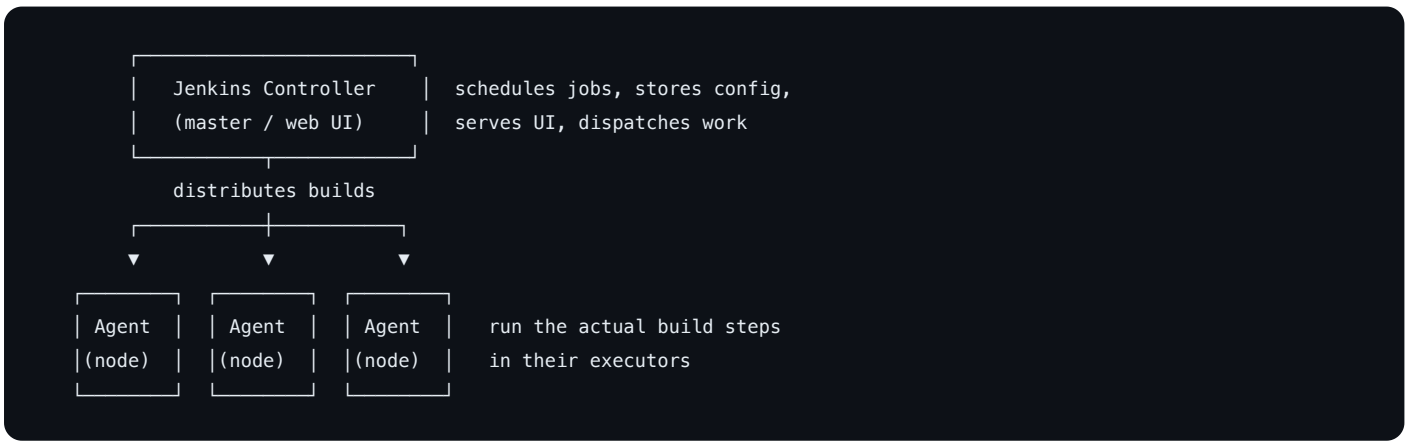
> 💡 **Mental model:** Jenkins is a **programmable robot butler** that watches your repos and runs your build/test/deploy steps on demand. You own the server (and its upkeep) — maximum flexibility, more operational responsibility than hosted CI.

Common uses: CI builds & tests on every commit, packaging artifacts/images, deploying to environments, scheduled/cron jobs, orchestrating complex multi-stage release pipelines.

---

2. 🏛️ Architecture (Controller & Agents)

Jenkins uses a **controller + agents** model:



Component	Role
Controller (formerly "master")	Brains: schedules jobs, manages config/plugins, serves the web UI, dispatches builds. Shouldn't run heavy builds itself.
Agent (node)	A machine that **executes** build jobs. Connects to the controller (via SSH, JNLP, or as a container).
Executor	A slot on an agent that runs **one build at a time** (an agent has N executors = N parallel builds).

3. 🧩 Core Concepts

Concept	Description
Job / Project	A configured task (build/test/deploy).
Build	One execution/run of a job (numbered #1, #2…).
Pipeline	A suite of jobs/stages defined as code (a `Jenkinsfile`).
Stage	A logical phase of a pipeline (Build, Test, Deploy).
Step	A single action within a stage (`sh 'make'`).
Node / Agent	Where a build runs.
Workspace	The directory on an agent where a build's files live.
Artifact	A build output you archive/publish.
Plugin	Add-on extending Jenkins functionality.

4. 🆚 Freestyle vs Pipeline

	Freestyle job	Pipeline
Defined via	Web UI checkboxes/forms	Code (a `Jenkinsfile`)
Versioned in SCM	❌ Not easily	✅ Yes (pipeline as code)
Complex logic	Limited	Full (stages, parallel, conditions, loops)
Best for	Simple, one-off tasks	Real CI/CD, modern usage

> ✅ Prefer **Pipeline** (a `Jenkinsfile` in the repo) — it's versioned, reviewable, and reproducible. Freestyle is legacy/simple-task only.

5. 📝 The Jenkinsfile (Declarative vs Scripted)

A `Jenkinsfile` lives in your repo and defines the pipeline as code. Two syntaxes:

	Declarative	Scripted
Style	Structured, opinionated (`pipeline { }`)	Full Groovy programming
Readability	Easier, recommended	More powerful, more complex
Use	Most pipelines	Advanced/dynamic logic

- **Declarative** is the modern default: a clear `pipeline { agent / stages / steps }` structure with built-in `post` , `environment` , `when` , `parallel` .
- **Scripted** is full Groovy — flexible but verbose; reach for it only when declarative can't express your logic.

---

6. ⚙️ Pipeline Example

A declarative `Jenkinsfile` — build → test → deploy:

```
pipeline {
  agent any
  environment {
    REGISTRY = 'ghcr.io/myorg/app'
  }
  stages {
    stage('Build') {
      steps { sh 'npm ci && npm run build' }
    }
    stage('Test') {
      steps { sh 'npm test' }
      post { always { junit 'reports/*.xml' } }   // publish test results
    }
    stage('Deploy') {
      when { branch 'main' }                      // only on main
      steps {
        withCredentials([string(credentialsId: 'deploy-token', variable: 'TOKEN')]) {
          sh './deploy.sh'
        }
      }
    }
  }
  post {
    success { echo 'Pipeline succeeded' }
    failure { echo 'Pipeline failed' }            // notify on failure
  }
}
```

> Key blocks: **`agent`** (where it runs), **`stages` / `steps`** (the work), **`when`** (conditional stages), **`environment`** (vars), **`post`** (always/success/failure handlers), **`parallel`** (run stages concurrently).

---

7. 🖥️ Agents & Distributed Builds

- The controller **distributes builds to agents** so work runs in parallel and on the right platform (Linux/Windows/macOS, GPU, etc.).
- **Agent connection methods:** SSH, inbound (JNLP), or dynamically provisioned (Docker/Kubernetes).
- **Label** agents (e.g. `linux`, `docker`) and target them in the pipeline (`agent { label 'linux' }`).
- **Dynamic agents** — the **Kubernetes plugin** spins up a fresh pod per build (clean, scalable, ephemeral), then tears it down.

---

8. ⏰ Triggers

How a build starts:

Trigger	How
SCM polling	Jenkins periodically checks the repo for changes (`pollSCM`).
Webhook	The SCM (GitHub/GitLab) pushes an event to Jenkins → instant build (preferred over polling).
Scheduled (cron)	Time-based (`triggers { cron('H 2 * * *') }`).
Upstream/downstream	One job triggers another on completion.
Manual / parameterized	A user clicks "Build" (optionally with parameters).

> Use **webhooks** over polling — instant builds and no wasted SCM checks. `H` in cron spreads load (hashed), avoiding the "everything at midnight" stampede.

---

9. 🔌 Plugins

- Jenkins' power comes from **1,800+ plugins** — Git, Docker, Kubernetes, Pipeline, Blue Ocean (modern UI), Credentials, Slack, JUnit, SonarQube, cloud agents, and more.
- Plugins extend triggers, build steps, SCM, UI, notifications, and integrations.
- ⚠️ Plugins are also the main **maintenance and security burden** — keep them (and the core) **updated**, and only install what you need (each adds attack surface and upgrade risk).

---

10. 🔐 Credentials & Security

- **Credentials store** — keep secrets (tokens, SSH keys, passwords, cloud creds) in Jenkins' credentials manager; reference by **ID** in pipelines via `withCredentials` / `credentials()`. Never hard-code secrets in a `Jenkinsfile`.
- **Folder/scoped credentials** limit who/what can use a secret.
- **Security basics:** enable authentication, use **role-based access control** (Matrix/RBAC plugin), keep Jenkins **off the public internet** (or behind a proxy/VPN), run builds on **agents not the controller**, and **patch core + plugins** regularly.
- Treat the **controller as highly privileged** — anyone who can run a pipeline can often run code on it.

11. ⚖️ Jenkins vs GitHub Actions vs GitLab

	Jenkins	GitHub Actions	GitLab CI/CD
Hosting	Self-hosted (you run it)	Hosted by GitHub (or self-hosted runners)	Hosted or self-managed
Config	`Jenkinsfile` (Groovy)	`.github/workflows/*.yml`	`.gitlab-ci.yml`
Setup/upkeep	You maintain server + plugins	Minimal (managed)	Minimal (if SaaS)
Flexibility	Maximum (plugins, any tool)	Large marketplace	Strong, integrated
Best for	Full control, complex/legacy, on-prem	Repos already on GitHub	Repos already on GitLab

> Jenkins = **maximum control + responsibility** (you own the server). Hosted CI (Actions/GitLab) = **less setup**, tightly integrated with the repo platform.

12. ✅ Best Practices

- **Pipeline as code** — keep a `Jenkinsfile` in the repo (declarative); avoid clicky Freestyle jobs.
- **Don't build on the controller** — run all builds on **agents**; keep the controller for orchestration.
- **Use webhooks**, not SCM polling, for instant builds.
- **Store secrets in the credentials store**; reference by ID, never hard-code.
- **Keep core + plugins updated**; install only what you need.
- **Ephemeral agents** (Docker/Kubernetes) for clean, scalable, reproducible builds.
- **Fail fast + publish results** (`junit` , archive artifacts); notify on failure (`post`).
- **Back up** `JENKINS_HOME` (config, jobs, credentials); treat Jenkins config as code where possible (JCasC).
- **Lock down access** with authentication + RBAC; keep it off the public internet.

13. 🧠 Quick Mental Model

- **Jenkins = self-hosted automation server for CI/CD** — you run it, you own it.
- **Controller** schedules + serves UI; **agents** run builds in **executors** (parallel slots).
- Modern usage = **Pipeline as code** in a `Jenkinsfile` (prefer **declarative**) — versioned & reviewable.
- A pipeline = **stages** → **steps**, with `agent` , `when` , `environment` , `post` , `parallel` .
- Triggered by **webhooks** (best), polling, cron, or upstream jobs.
- **Plugins** give it superpowers — and are its main upkeep/security burden; keep them patched.
- Secrets live in the **credentials store**; run builds on **agents**, not the controller; lock it down.
- vs hosted CI: **more control, more maintenance.**

14. ❓ Common Interview Questions

Rapid-fire questions interviewers ask about Jenkins:

- **Q: What is Jenkins?** — An open-source, self-hosted automation server for CI/CD; you define pipelines as code and it builds/tests/deploys on triggers.
- **Q: Controller vs agent?** — The controller schedules jobs, stores config, and serves the UI; agents (nodes) actually execute the builds. Don't run heavy builds on the controller.
- **Q: What is an executor?** — A slot on an agent that runs one build at a time; N executors = N concurrent builds on that agent.
- **Q: Freestyle vs Pipeline?** — Freestyle is UI-configured and not versioned; Pipeline is code (`Jenkinsfile`), versioned in SCM, supports complex logic — the modern choice.
- **Q: Declarative vs Scripted pipeline?** — Declarative is structured/opinionated and recommended; Scripted is full Groovy for advanced/dynamic needs.
- **Q: How do you trigger a build?** — SCM webhook (preferred), SCM polling, cron schedule, upstream job, or manual/parameterized.
- **Q: How are secrets handled?** — In the Jenkins credentials store, referenced by ID via `withCredentials` / `credentials()` — never hard-coded.
- **Q: How do you scale Jenkins?** — Add agents; use dynamic ephemeral agents (Docker/Kubernetes) that spin up per build.
- **Q: Jenkins vs GitHub Actions?** — Jenkins is self-hosted with maximum flexibility (and upkeep); Actions is managed and tightly integrated with GitHub repos.
- **Q: What is a Jenkinsfile?** — A text file in the repo defining the pipeline as code (stages/steps), enabling versioning and review.
- **Q: Why not build on the controller?** — Security and stability — builds run arbitrary code; isolate them on agents and keep the controller for orchestration.
